A.E ReasoningFlow annotation guide: Tool calls and web search

Proposed ReasoningFlow annotation supplement | Nathan Conklin | Version 0.2 | 12 October 2026

A.E0 Implementation update: Client-side annotation without an LLM

Design decision and OMI rationale

The revised implementation computes ReasoningFlow entirely on the client, without an LLM or any other model inference for annotation. The reason for this change is faster OMI feedback: the user should receive node-type coloring and selection feedback without the interface waiting on additional LLM calls. Local processing removes the annotation-request dependency while the primary chat continues to stream. Text rendering remains independent of annotation readiness, and hover and selection read the annotations already held in memory. Faster feedback is the design objective; no measured latency improvement is claimed here.

The original ReasoningFlow automatic pipeline uses rule-based coarse segmentation followed by LLM-based fine segmentation, node classification, a separate Conclusion update, and incoming-edge annotation for each destination node. This implementation retains the paper's vocabulary but replaces that automated annotation mechanism with deterministic parsing and conservative rules. The paper's annotation results do not validate this replacement. [1, §4.2; Appendix C.1; Appendix I]

Processing boundary and unchanged interface

Only ReasoningFlow annotation loses model inference. The application's primary chat generation, streamed `reasoning_content`, tool execution, and web-search behavior remain unchanged. Annotation uses ordinary text parsing, lexical comparisons, source mappings, and recorded execution events. It has no LLM fallback, embedding model, learned local classifier, or extra analysis request. Normal loading of saved conversation records is allowed; annotation must not rerun tools or retrieve missing pages.

The refactor preserves the existing Reasoning Card, its hover highlights, persistent sentence and multi-paragraph selections, mixed-type red outline, synchronized Legend readouts, and approved legend image. All 11 node types and 20 edge labels remain available. Their meanings, colors, box shapes, and line patterns do not change. Different rules may produce different labels or leave more text unclassified; this changes annotation coverage, not the interaction design. The in-memory structure remains named ReasoningFlow and remains reusable by the future Reasoning Tree view.

Deterministic node and relationship annotation

Known source identity establishes `ToolCall` for a recorded request, `ToolResult` for its observed response, and `Context` for traceable supplied evidence. Generated reasoning text is classified separately using guarded rules for intentions, evaluations, assumptions, answer assertions, and narrowly recognized restatements or derivations. Rules must account for context, exclusions, and quoted text. A keyword alone does not establish a role; for example, “I think the answer is 17” is not automatically `Reflection`. Unmatched text must not default to `Reasoning` or `Fact`.

Call identifiers support `returns`; extraction mappings support `provides`; resolved explicit source references support `cites`; recorded retry lineage supports `retry_of`. Inferred `invokes`, `grounded_in`, and original semantic edges require a documented rule and supporting evidence. Adjacency, a shared topic, or a citation alone must not create a semantic dependency. Preserve earlier-to-later endpoints and valid multiple labels between the same nodes. Keep uncertain relationships absent rather than forcing a connected graph.

A.E0 Client-side annotation without an LLM (continued)

Paragraph-triggered streaming and local reconciliation

Capture incoming text without blocking its display. When a paragraph or supported block is complete, segment it into sentence or justified clause spans, apply role rules, add evidenced relationships, and update the in-memory graph. Paragraphs are scheduling units; they are not automatically single nodes. Protect code and equation blocks, retain an unfinished tail, and preserve exact source offsets. Do not annotate on every token, pause, or mouse event. Structured tool records and directly evidenced associations can update as their events arrive. Use bounded batches or a client-side worker where needed so local analysis does not monopolize the interface thread.

At the authoritative interaction-completion boundary, process the final usable paragraph even without a trailing delimiter. Resolve remaining references, revisit rules whose context has become available, and validate span alignment, endpoint types, chronological availability, and acyclicity. Repair relationships affected by changed nodes; preserve unchanged recorded associations. A pause or tool handoff is not necessarily completion. Cancelled or failed interactions retain their incomplete status. Reconciliation must not import the separate final-answer card, manufacture a `Conclusion`, or attribute later evidence to an earlier statement.

This final pass is deterministic completion and consistency checking, not an independent semantic review. Repeating a rule with identical inputs does not correct its systematic errors. Publish a consistent graph revision and preserve selections through source-range mapping when nodes split, merge, or change type. Reject stale results from older generations; hover, clicks, dragging, and Legend display continue to use cached annotations without starting analysis.

First-view computation for older conversations

When an older conversation is selected and saved reasoning has loaded sufficiently for its existing Reasoning Card to be available for viewing, compute any missing ReasoningFlow using the same client-side rules. Process the available completed paragraphs in bounded batches and reconcile the loaded record without waiting for a live completion event. Preserve its original complete, incomplete, or unknown status. Use saved tool and citation records when available; missing records do not justify reconstructed executions, regenerated reasoning, or substituted final-answer text.

Reuse compatible in-memory results across repeat views and ordinary navigation. Key cached work to the owning conversation/card, source identity or revision, and rule version; deduplicate in-progress work and prevent results from being applied to a different conversation. Do not precompute the entire history library or overwrite saved source content. “First view” applies within the retained cache lifetime; a full application reload or eviction may require recomputation. No new persistent storage is introduced.

How to apply the existing annotation guide

The following label definitions, examples, endpoint restrictions, and legend remain the semantic contract. They define a valid annotation, not a guarantee that deterministic rules will identify every instance. In this implementation, “annotator” may refer to a rule-based annotator; `annotator_inferred` does not require an LLM. Retain the evidence-method distinctions already defined in this appendix, and identify the responsible rule and version in supporting metadata. Pending or unclassified text is a status, not an additional node type or color; it keeps the default appearance while remaining selectable.

The expected tradeoff is lower or different semantic coverage in exchange for removing additional model-call latency from OMI feedback. Conservative abstention is preferable to an unsupported color or relationship. Evaluate feedback delay, classification coverage, rule errors, and graph consistency on representative traces. This profile makes no claim of equivalence to the original LLM annotations, measured speedup, or recovery of the model's hidden computational path.

Design basis: ReasoningFlowClientSideRefactorPlanningPrompt.md, including first-view historical backfill (§§1, 3–10), and the OMI-feedback rationale requested in this conversation. This section supersedes earlier LLM-annotation requirements; the annotation taxonomy below is retained.

Purpose and relationship to the paper

ReasoningFlow annotates the discourse structure of an expressed reasoning trace with eight reasoning-node types, a Context category, and 14 semantic edge labels. Its Context definition already includes retrieved documents and tool responses. This supplement retains those definitions and adds ToolCall and ToolResult to represent the structured request and returned payload that occur between a plan and external evidence. Six added relationships connect expressed intent to invocation, calls to results, results to evidence spans, and evidence to later statements; they also record explicit citations and technical retries. Web search is a retrieval subtype of ToolCall. These additions produce a profile with 11 node types and 20 edge types. The operational and provenance extension is proposed in the accompanying design discussion; it was not introduced or evaluated by Lee et al. [1, §3; Appendix A; 2]

Scope of this supplement

Use this document with Appendix A of the supplied ReasoningFlow paper. The original node definitions appear in Table 3 and the original edge definitions in Table 4. Tables A.E1–A.E8 below add two node labels and six relationship labels in the same Label, Description, Subtype, and Example format. Original labels retain their source definitions and color coding. New labels are marked as additions. Examples in this supplement are synthetic annotation examples, not records of real searches or observed model behavior.

Annotation unit and source boundary

An original reasoning node remains a contiguous, nonoverlapping text span in its own source message. ToolCall and ToolResult are structured event records rather than sentence-sized reasoning units. Context nodes can reference exact spans or traceable extractions from a ToolResult. The result envelope and its Context spans occupy different representational layers; their shared bytes are not duplicated reasoning steps. Preserve original messages and payloads, and store source IDs, offsets or JSON pointers, and representation hashes.

Observed and inferred information

Call/result identifiers can establish an execution association. A prose statement that the model will search cannot establish that the call happened. Likewise, a result can exist without being shown to the model, and an explicit citation can exist without supporting the cited claim. Record the evidence method for each edge: runtime_observed, span_derived, explicit_reference, annotator_inferred, or human_reviewed. Human review should retain the original evidence method. These methods are metadata, not new node or relationship labels.

Compact annotation procedure

Begin with the recorded event sequence and the context manifest for the relevant model invocation. Identify actual calls and results, then extract eligible Context spans. Segment and classify the exposed text using the original guide. Add recorded operational links before inferring intent or evidence dependence. Apply the endpoint rules in this supplement, record supporting spans or event IDs, and leave unsupported links absent. During streaming, incomplete units and unresolved associations remain provisional. Reconcile them after the episode closes without inventing missing reasoning.

A.E1 Approved legend and directional conventions

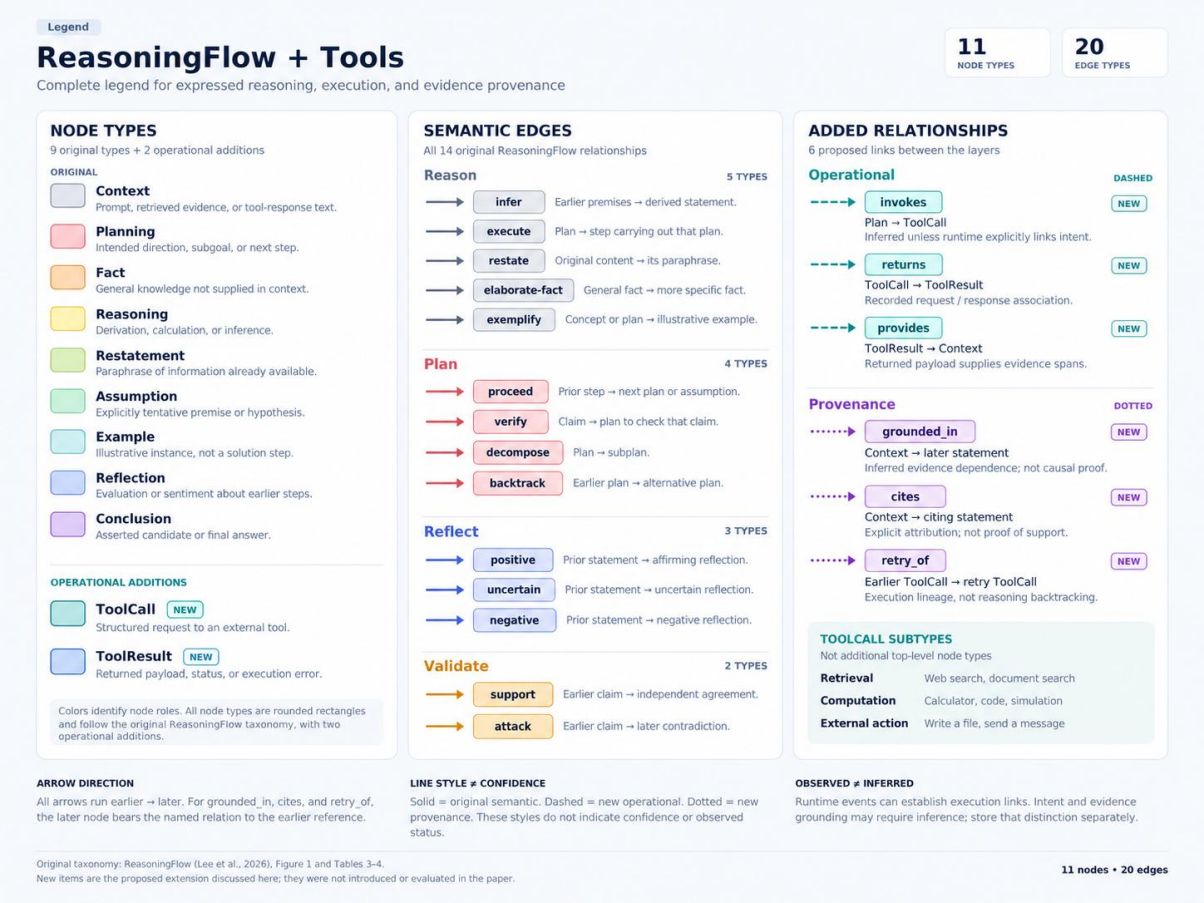


Figure A.E1: Approved legend from the design discussion. The operational and provenance additions are proposed, not part of the original paper. [2]

The graph stores earlier → later endpoints. The names grounded_in, cites, and retry_of read from the later endpoint back to the earlier reference. The table below specifies that convention explicitly.

Label	Stored source → target	Interpretation
`invokes`	Planning → ToolCall	The earlier plan is operationalized by the later call.
`returns`	ToolCall → ToolResult	The earlier call has the later recorded result.
`provides`	ToolResult → Context	The earlier result supplies the later evidence representation.
`grounded_in`	Context → statement	The later statement is grounded in the earlier Context.
`cites`	Context → citing statement	The later statement cites the earlier Context/source.
`retry_of`	earlier ToolCall → later ToolCall	The later call is a retry of the earlier call.

All nodes remain boxes. Original semantic links are solid, added operational links are teal dashed, and added provenance links are purple dotted, as shown in the approved image. Line pattern indicates family, not confidence or observed status.

A.E2.1 ToolCall

A ToolCall records a structured request, including requests that are still being assembled or were denied before dispatch. Annotate the observed lifecycle separately from the request's role. A ToolCall does not by itself assert that execution started or completed.

Label	Description	Subtype	Example
`ToolCall` **NEW**	ToolCall is a recorded invocation request to an external tool, API, computation facility, or action executor. Keep one stable node per call attempt, using the provider/orchestrator call identifier and attempt identifier. Preserve the tool name, structured arguments, request channel, timestamps, and observed lifecycle. Fragmented arguments update the same provisional node. A text statement such as "I will search" remains Planning unless a structured request is recorded. The subtype families are retrieval, computation, and external_action; operation refinements do not create new top-level types. An observed request can end as denied, canceled, failed, or outcome_unknown. The graph must not report dispatch or success without corresponding evidence.	Retrieval: web search	`ToolCall` *T1* *Recorded request: web_search(query="DemoCache default timeout").* *call_id=search-1; attempt_id=1.* *This node denotes the request, not the search hits.*
		Retrieval: opening a source	`ToolCall` *T2* *Recorded request: document_fetch(source_ref="S1").* *Opening the page is a separate call from the search that discovered its URL.*
		Retrieval: local or structured search	`ToolCall` *T3* *Recorded request: file_search(query="deployment timeout") or database_query(query_id="approved-read-7").* *Preserve the actual tool identity and arguments.*
		Computation	`ToolCall` *T4* *Recorded request: calculator(expression="12 * 7").* *The operation is computation; the returned value belongs in ToolResult.*
		External action	`ToolCall` *T5* *Recorded request: write_file(path="report.md", content_ref="draft-1").* *A request to write a file is not proof the file was written.*
		Opaque hosted operation	`ToolCall` *T6* *A provider exposes one hosted-search operation but no subqueries.* *Record the visible operation scope; do not create imagined page-fetch calls.*

Table A.E1: Proposed annotation guide for the ToolCall node.

Required evidence. A structured request/event or a human-verified execution record. A model sentence alone is insufficient.

Boundary cases. Tool name unknown but operation identity recorded: keep the available identifier and mark missing fields. No exposed plan: retain an unlinked ToolCall. Partial argument fragments: update the same node rather than adding one node per fragment.

Do not confuse with. Planning states an intention; ToolCall records a request. ToolCall retry attempts have distinct attempt IDs and may later connect through retry_of.

A.E2.2 ToolResult

A ToolResult preserves the returned envelope, status, or error for a recorded operation. It can contain useful evidence, an empty payload, or an unsuccessful outcome. Evidence text extracted from it is represented as Context.

Label	Description	Subtype	Example
`ToolResult` **NEW**	ToolResult is an observed response item associated with a ToolCall. It can hold structured data, text, a returned file reference, an acknowledgment, an error, or an explicitly returned empty result. Preserve call_id, attempt_id, result_item_id, producer identity, payload reference, status, and available timestamps. Streaming payload fragments update their result item; they are not independent reasoning units. A call can have several distinct returned items. A result's existence does not certify truth, successful execution, or delivery to the model. If a runtime emits a timeout report, identify the runtime as producer rather than presenting it as a tool-server response. If no result is observed, leave the outcome unknown instead of fabricating a result.	Search-result collection	`ToolResult` *U1* *Returned list: [{source_id:"S1", snippet:"Default timeout: 20 seconds."}].* *The result may provide separate Context nodes for different hits.*
		Document or record content	`ToolResult` *U2* *Returned manual text plus retrieval status.* *Preserve the source reference and representation; full text and a search snippet are distinct payloads.*
		Computed output	`ToolResult` *U3* *calculator returns value=84.* *The value can supply Context; a later model derivation remains Reasoning.*
		Action acknowledgment or artifact	`ToolResult` *U4* *write_file returns status="saved", artifact_ref="file-7".* *This reports the executor acknowledgment. Artifact contents become Context only if available.*
		Execution or runtime error	`ToolResult` *U5* *fetch returns status="timeout", producer_kind="runtime".* *The error describes this attempt and does not prove the requested page does not exist.*
		Empty or partial response	`ToolResult` *U6* *Search returns hits=[]; or fetch returns a partial text body.* *Record empty/partial status and annotate only the returned content.*

Table A.E2: Proposed annotation guide for the ToolResult node.

Required evidence. A recorded returned item or error event and its actual correlation data. A missing return is not an implicit ToolResult.

Context extraction. A provides edge links the envelope to a quoted span or traceable extracted representation. Preserve offsets or a JSON pointer and representation identity.

Do not confuse with. ToolResult is the operational envelope. Context is its evidence content. “No hits” is a property of one query response, not a universal absence claim.

A.E3.1 invokes

Use invokes to connect an expressed plan to the external request that implements that plan. The call's occurrence and the attribution of intent are separate pieces of evidence.

Label	Description	Subtype	Example
`invokes` **NEW**	invokes connects a Planning node to a later ToolCall whose requested operation implements that plan. The source must contain the expressed plan; the target must be an observed call. Confirm the relation through an explicit runtime mapping when available, or annotate it from the plan and tool arguments. Temporal proximity alone is insufficient. One plan may invoke several calls; more than one explicit plan may be relevant to a call if each relation is justified. Use no synthetic Planning node when reasoning is unavailable. A denied or canceled request can still implement an expressed intention at the request level; its lifecycle remains visible on ToolCall. invokes is an added operational relationship and does not replace the original execute edge between a plan and a text step.	Searching for evidence	`Planning` *P1: "I should search for the default timeout."* → *invokes* → `ToolCall` *T1: web_search(query="DemoCache default timeout").* *Basis: annotator_inferred unless explicitly mapped by the runtime.*
		Fetching a selected source	`Planning` *P2: "I should open the manual to verify this."* → *invokes* → `ToolCall` *T2: document_fetch(source_ref="manual-S1").* *The selected source matches the stated verification plan.*
		Executing a calculation	`Planning` *P3: "I will calculate twelve times seven."* → *invokes* → `ToolCall` *T3: calculator(expression="12 * 7").* *The subsequent value is recorded separately.*
		Requesting an external action	`Planning` *P4: "I will save this draft to report.md."* → *invokes* → `ToolCall` *T4: write_file(path="report.md").* *The edge does not certify the write succeeded.*
		Parallel implementation of one plan	`Planning` *P5: "I should compare the two manuals."* → *invokes* → *T5: fetch(manual-A); and* → *invokes* → *T6: fetch(manual-B).* *Annotate two edges; no ordering edge is needed between T5 and T6.*

Table A.E3: Proposed annotation guide for invokes.

Endpoint rule. Planning → ToolCall. This version does not use Reasoning → ToolCall as a substitute for an unexpressed plan.

Evidence rule. Use runtime_observed only when the runtime explicitly identifies the source plan. Otherwise use annotator_inferred with supporting text and arguments.

Negative example. "I should search" followed by no recorded request has no invokes edge. A nearby unrelated calculator call must not be attached to a search plan.

A.E3.2 returns

returns records request/response association. Match identifiers and scope before adding the edge; do not match parallel operations by visual or arrival order.

Label	Description	Subtype	Example
`returns` **NEW**	returns connects a ToolCall to its observed ToolResult. The call and result must share a recorded call identifier, attempt identifier, or an equivalent verified correlation record. The edge is valid for successful payloads, empty responses, action acknowledgments, and explicit errors. It does not express semantic support or guarantee success. Distinct result items can create several returns edges from one call; network fragments of the same result item update that item instead. If correlation is unresolved, retain the result with unresolved-association metadata and postpone the edge. A tool request that has not produced a return has no returns edge. Out-of-order receipt does not change which call owns a result.	Successful retrieval	`ToolCall` *T1: call_id=search-1.* *→ returns →* `ToolResult` *U1: call_id=search-1; hits=[S1,S2].* *Basis: recorded request/result association.*
		Computed output	`ToolCall` *T2: call_id=calc-1; expression="12 * 7".* *→ returns →* `ToolResult` *U2: call_id=calc-1; value=84.* *The edge identifies the returned item, not a reasoning inference.*
		Error or empty response	`ToolCall` *T3: call_id=fetch-2; attempt=1.* *→ returns →* `ToolResult` *U3: call_id=fetch-2; attempt=1; status="timeout".* *An explicitly returned hits=[] response is also a valid result.*
		Parallel calls and multiple items	*T4 and T5 run concurrently. U5 arrives first with call_id=T5.* *Annotate T5 → returns → U5, then T4 → returns → U4.* *If T4 returns two distinct item IDs, both may have T4 as source.*

Table A.E4: Proposed annotation guide for returns.

Endpoint rule. ToolCall → ToolResult. A ToolResult normally has one owning call attempt; a combined provider result must preserve the correlation scope actually exposed.

Evidence rule. runtime_observed with the correlation event IDs. An annotator may verify identifiers but must not invent them from content similarity.

Negative example. A transport disconnect without a returned error does not justify a synthetic failure result. Record unknown outcome on the call and await further evidence.

A.E3.3 provides

provides preserves the path from a returned payload to an inspectable evidence span. It records extraction provenance, not whether the model accepted or used the content.

Label	Description	Subtype	Example
`provides` **NEW**	provides connects a ToolResult to Context derived from that result. Context may be an exact text span, a structured field, an excerpt, or a traceable extracted representation. Record the original payload identity and hash, plus offsets, a JSON pointer, or a documented representation mapping. Several Context nodes may come from one result. The target must not add claims absent from the source. If extraction transforms the representation, retain the transformation identity and the extracted representation's hash. Context can be visible in the graph before it is delivered to the model; the context manifest determines whether it is eligible as evidence for later model statements. A provides edge alone is not evidence of model access, use, or source credibility.	Individual search hit	`ToolResult` *U1: search list includes S1.snippet.* → *provides* → `Context` *C1: "Default timeout: 20 seconds."* *Locator: /hits/0/snippet in the recorded result.*
		Document excerpt	`ToolResult` *U2: returned manual body.* → *provides* → `Context` *C2: "The default timeout is 30 seconds."* *Preserve start/end offsets in the identified text representation.*
		Structured computation result	`ToolResult` *U3: {"value":84}.* → *provides* → `Context` *C3: value=84.* *Locator: /value. A later explanation of the arithmetic is not part of this extraction.*
		Traceable transformed text	`ToolResult` *U4: HTML response payload.* → *provides* → `Context` *C4: extracted paragraph in text representation R4.* *Store the original hash, extraction method, and R4 hash.*
		Observed empty-result statement	`ToolResult` *U5: hits=[].* → *provides* → `Context` *C5: "This query returned zero hits."* *This describes U5; it must not be rewritten as "No relevant documents exist."*

Table A.E5: Proposed annotation guide for provides.

Endpoint rule. ToolResult → Context. User-supplied Context needs no provides edge unless it was itself delivered through a recorded tool result.

Evidence rule. span_derived for verified extraction, with source location and transformation metadata. Never treat a generated summary as an exact source span.

Negative example. A result listing a URL does not provide the unseen full text of that page. Record the search snippet; add full-document Context only after an observed fetch or another available source.

A.E3.4 grounded_in

grounded_in attributes information dependence to an earlier Context span. It is a semantic/provenance judgment and requires more than a shared topic or an explicit citation marker.

Label	Description	Subtype	Example
`grounded_in` **NEW**	grounded_in connects earlier Context to a later original text-role node whose content depends on information in that Context. Read the label from the target: the later statement is grounded in the earlier context. Require that the context was available to the relevant model invocation and that an evidence span explains the attribution. The target may be a paraphrase, derived claim, assumption, plan, reflection, example, or conclusion when its dependence is supported. Fact is excluded in this profile because the original Fact category is independent of supplied context; reassess a purported grounded Fact as Restatement or Reasoning. The edge can coexist with infer or restate because it records evidence provenance. It does not certify correctness, source reliability, or hidden mechanistic dependence.	Source-based answer	`Context` *C1: "The default timeout is 30 seconds."* → *grounded_in* → `Conclusion` *K1: "The default timeout is 30 seconds."* *The matched span and input manifest support evidence attribution.*
		Derivation using retrieved values	`Context` *C2: "A batch has 12 items."* *and C3: "Each item weighs 7 units."* *Both → grounded_in →* `Reasoning` *R1: "The batch weighs 84 units."* *Add appropriate infer links independently.*
		Source-based planning or reflection	`Context` *C4: "Full details are in the installation manual."* → *grounded_in* → `Planning` *P1: "I should read that manual."* *The source informs the expressed plan; invokes requires a separate call.*
		Conditional evidence use	`Context` *C5: "The estimate assumes constant load."* → *grounded_in* → `Assumption` *A1: "I will assume constant load for this estimate."* *The assumption remains conditional despite having a source.*
		Use of an error or empty result	`Context` *C6: "This fetch attempt timed out."* → *grounded_in* → `Reflection` *F1: "I cannot verify the page from this attempt."* *The scope is this attempt, not the existence of the page.*

Table A.E6: Proposed annotation guide for grounded_in.

Endpoint rule. Context → eligible later text-role node. ToolCall, ToolResult, Context, and context-independent Fact are not targets in this version.

Evidence rule. Normally annotator_inferred, optionally human-reviewed, with matched spans and availability evidence. Numeric confidence is optional and must not be treated as calibrated by default.

Negative example. A result in the log but omitted from the model input is not a grounding source. A citation to a page with unrelated text supports cites only, not grounded_in.

A.E3.5 cites

cites records explicit source attribution. Annotate the reference that is present, even when the source does not support the claim; evaluate support separately.

Label	Description	Subtype	Example
`cites` **NEW**	cites connects earlier Context to a later text-role node that explicitly references that context or its source. Read the label from the target: the later statement cites the earlier context. References can be provider citation markers, numbered references, explicit source IDs, URLs, or other unambiguous attribution. Resolve the reference against the actual available source records and preserve its scope, such as a clause or sentence. Citation by itself does not entail grounded_in, infer, or support. An explicit citation can be attached to a parametric Fact without changing the Fact's role if the citation is attribution rather than evidence that the fact was derived from context. If the source is unavailable or ambiguous, retain unresolved citation metadata instead of inventing source content or forcing an edge.	Resolved source marker	`Context` *C1: source_id=S2 with an available manual excerpt.* *→ cites →* `Conclusion` *K1: "The default timeout is 30 seconds [S2]."* *Store the marker and its target text span.*
		Explicit source attribution	`Context` *C2: retrieved record with source_id=manual-7.* *→ cites →* `Restatement` *R1: "According to manual-7, the default is 30 seconds."* *The record ID is explicit and unambiguous.*
		Several cited sources	`Context` *C3 (S3) and C4 (S4)* *Both → cites →* `Reasoning` *R2: "Both manuals specify the same value [S3, S4]."* *Resolve and annotate each citation independently.*
		Citation without support	`Context` *C5: a source about installation paths.* *→ cites →* `Conclusion` *K2: "The timeout is 30 seconds [S5]."* *The citation is explicit, but its content does not justify grounded_in or support.*
		Delayed citation metadata	*The provider emits a reference mapping after the answer. If the referenced Context was available before the statement, resolve the cites edge using that logical order.* *Otherwise keep the reference unresolved; do not invent earlier access.*

Table A.E7: Proposed annotation guide for cites.

Endpoint rule. Context → later original text-role node with an explicit reference. The marker may be in reasoning text or the user-visible answer; preserve the channel.

Evidence rule. explicit_reference with a citation locator and source-resolution record. Human review can resolve an ambiguous reference but should record the original ambiguity.

Negative example. Two statements using the same terminology do not establish citation. A naked URL with no recoverable source record is reference metadata, not an invented Context containing unseen page text.

A.E3.6 retry_of

retry_of describes operational repetition of the same objective. It is separate from the model's change of plan, which the original schema can represent with backtrack.

Label	Description	Subtype	Example
`retry_of` **NEW**	retry_of connects an earlier ToolCall attempt to a later ToolCall that retries that operation for the same objective. Read the label from the target: the later call is a retry of the earlier call. Prefer an explicit retry parent, attempt counter, or orchestration record. Arguments may remain identical or change slightly to repair an execution-level problem, such as valid request formatting or a permitted timeout adjustment. A new investigation, independent verification, changed hypothesis, or different search strategy is not automatically a retry. Annotate the immediately preceding retry attempt rather than every pair in a chain. Store original objective and retry lineage as metadata. A retry request need not succeed, and its status is independent of this edge.	Transient execution failure	`ToolCall` *T1: fetch manual; attempt=1; observed timeout.* → *retry_of* → `ToolCall` *T2: same fetch; attempt=2; retry_parent=T1.* *A result from T1 may remain separately attached through returns.*
		Request repair	`ToolCall` *T3: query rejected because a required format field is missing.* → *retry_of* → `ToolCall` *T4: repaired request for the same query objective, with recorded retry lineage.* *A different information objective would require separate review.*
		Recorded backoff retry	`ToolCall` *T5: search request receives a retryable runtime status.* → *retry_of* → `ToolCall` *T6: orchestrator reissues the same objective after backoff.* *Use the recorded runtime association.*
		Retry chain	*T7 attempt=1 → retry_of → T8 attempt=2 → retry_of → T9 attempt=3. Do not add T7 → T9 merely because both belong to the same retry chain. The chain root can be stored as metadata.*

Table A.E8: Proposed annotation guide for retry_of.

Endpoint rule. Earlier ToolCall → later ToolCall. A self-loop is invalid. A result is not the source of retry_of, though its error may explain the retry in metadata.

Evidence rule. runtime_observed when retry lineage is recorded; otherwise an explicitly reviewed inference with its basis. Similar arguments alone are insufficient.

Negative example. “That evidence is weak; I will search a different archive” introduces an alternative Planning node. Use backtrack between plans where appropriate, then invokes to the new call; do not force retry_of.

A.E4 Compatibility with the original semantic guide

The new records surround the original discourse annotation rather than replacing it. Apply the original role definitions to text about a tool, and apply the operational labels to recorded requests and responses. The distinctions below are adapted from Appendix A of [1] and constrained by this proposed extension.

Distinction	Annotation rule
Context / Fact	A returned manual paragraph is Context. A model paraphrase is Restatement; a deduction from it is Reasoning. Fact remains knowledge independent of supplied context.
execute / invokes	Planning → execute → a text step that implements the plan. Planning → invokes → a recorded external request. A tool dispatch is not a Reasoning node.
restate / grounded_in	Context → restate → a faithful paraphrase records the discourse relation. A coexisting grounded_in edge records evidence provenance. Neither is implied by mere topic similarity.
infer / provides	Context → infer → a derived claim. ToolResult → provides → a span in Context. Extraction is not logical inference.
verify / returns	Claim → verify → a plan to check it. A tool returning data only creates returns; verification has not been established solely by a successful response.
support / cites	Earlier claim → support → an independently derived consistent claim. Context → cites → an explicit source reference. A source URL is not a support relation.
attack / negative	An explicit contradictory answer or correction can receive attack from the earlier claim. “This seems wrong” is a Reflection and may receive negative instead.
backtrack / retry_of	A new reasoning plan can backtrack from an earlier plan. A technical reattempt of the same operation can be retry_of an earlier ToolCall.
proceed / timeline order	proceed leads to a Planning or Assumption node under the original guide. Do not use it to connect every adjacent event or to hide a missing operational relationship.
Conclusion / finality	Candidate answers can be Conclusion during streaming. Finality is episode metadata. An interrupted trace does not require a fabricated Conclusion.

Multiple valid links can coexist between a source and destination. For example, a manual excerpt can be connected to a final answer by infer, grounded_in, and cites when each definition is satisfied. The three labels record different relationships. Do not generate the extra edges by implication or remove them as duplicate endpoint pairs.

For support and attack, preserve the original direction from an earlier proposition to a later independent agreement or contradiction. Compare conclusions only when they address the same question. A cited document may be unreliable; the citation remains an explicit attribution while its content and the later reasoning require separate evaluation.

Evidence-path displays must select relationship types deliberately. A path that reaches a final answer through attack does not make the earlier wrong claim a supporting premise. A returns/provides chain establishes execution and extraction provenance, not a logical derivation.

A.E5.1 Worked example: search, verification, and correction

DemoCache and the values in this example are invented. The search snippet states a timeout of 20 seconds; the opened manual states 30 seconds. The input manifest records that each Context span was available before the associated later statement. These are instructional examples, not a factual claim about a real product.

ID	Label	Event or exact text
P1	Planning	"I should search for the DemoCache default timeout."
T1	ToolCall	web_search; call_id=search-1; query=DemoCache default timeout.
U1	ToolResult	Observed search list containing S1.
C1	Context	S1 snippet: "Default timeout: 20 seconds." Supplied to the model.
K1	Conclusion	"The default timeout is 20 seconds." Candidate answer.
F1	Reflection	"That answer may depend on an outdated snippet."
P2	Planning	"I should check the manual itself."
T2	ToolCall	document_fetch; call_id=fetch-1; selected manual source.
U2	ToolResult	Observed document-fetch payload and status.
C2	Context	S2 manual: "The default timeout is 30 seconds." Supplied to the model.
R1	Restatement	"The manual specifies a default timeout of 30 seconds."
K2	Conclusion	"The default timeout is 30 seconds [S2]." Final answer for this episode.

Table A.E9: Synthetic node annotations for the search-and-correction episode.

K1 is an asserted candidate answer and is therefore Conclusion. If the model had only said "The snippet reports 20 seconds," that sentence would be Restatement and would remain true as a report of the snippet. The later manual would not contradict that historical report; the attack relation below applies to the actual candidate answer.

R1 restates C2. Under the original premise-selection guidance, the final answer can cite the original C2 rather than routing its premise through a duplicate restatement. S2 identifies the full-document evidence, not merely the earlier search hit. [1, Appendix A, infer and restate]

A.E5.2 Worked example: relationship annotations

The following set illustrates the relevant relationship types. It is not an experimentally adjudicated exhaustive graph. Additional links require their own source evidence and must satisfy the original or extended definition.

Endpoints	Label	Basis
P1 → T1	`invokes`	Inferred plan/request alignment unless runtime explicitly maps P1.
T1 → U1	`returns`	Recorded call_id=search-1 association.
U1 → C1	`provides`	Exact search-snippet locator and representation hash.
C1 → K1	`infer`	Candidate answer derived from the snippet; correctness is separate.
K1 → F1	`uncertain`	The model expresses uncertainty about its candidate answer.
K1 → P2	`verify`	The later plan checks the earlier candidate claim.
F1 → P2	`proceed`	The uncertainty motivates the next plan.
P2 → T2	`invokes`	The manual fetch implements the expressed verification plan.
T2 → U2	`returns`	Recorded call_id=fetch-1 association.
U2 → C2	`provides`	Traceable manual excerpt.
C2 → R1	`restate`	A paraphrase of the available manual text.
C2 → K2	`infer`	The original evidence, rather than duplicate R1, supplies the premise.
C2 → K2	`grounded_in`	Evidence attribution backed by the matched manual span.
C2 → K2	`cites`	Explicit S2 source marker.
K1 → K2	`attack`	The later answer contradicts the earlier candidate answer.

Table A.E10: Illustrative edges for Table A.E9. The same endpoints can carry more than one label.

If the manual fetch times out and the orchestrator records a second attempt T3 for the same objective, add T2 → retry_of → T3. Preserve the first attempt's observed error result. If the model instead changes its search strategy, annotate the new Planning node and use backtrack where the original guide applies; similar queries alone do not establish a retry.

A recorded result may support extraction before semantic interpretation is available. The streaming interface can therefore show returns and provides while infer, grounded_in, or invokes remain unresolved. No fabricated plan or evidence-use edge is needed to make the partial graph connected.

A.E6 Streaming annotation and adjudication

Annotate a stable text unit only after enough content exists to establish its boundaries and function. Keep an unfinished tail and do not use API chunk boundaries as sentence boundaries. Earlier annotations may be revised after more text or metadata arrives. Revision history belongs in metadata; it does not add relationship types. [1, Appendix C.1; Appendix I; 2]

Case	Decision
No exposed reasoning	Annotate recorded ToolCall, ToolResult, and available Context. Do not generate a synthetic plan or reasoning trace. Mark the disclosure mode.
Plan but no call	Retain Planning. There is no ToolCall or invokes edge without a recorded request.
Arguments still streaming	Keep one provisional ToolCall and append fragments. Do not infer dispatch from an incomplete request.
Tool result absent	Use unknown/pending/canceled status as supported by events. Do not fabricate a returned payload.
Evidence not delivered	provides may record extraction, but infer and grounded_in must not imply access by a model invocation that never received it.
Source reference unresolved	Keep the literal citation and its location as unresolved metadata. Do not fill missing content from memory.
Repeated query	Check objective and retry lineage. Repetition alone does not distinguish retry, verification, or a new search strategy.
Partial page or snippet	Annotate only observed content and its representation. Do not infer full-document claims from a snippet.
Late metadata	Use recorded logical availability when supported. Otherwise keep the candidate unresolved rather than drawing a backward edge.
Changed segmentation	Preserve the original source and offsets. Retire old node/edge versions atomically and recompute affected relations.
Tool action succeeded?	A request or dispatch is not success. Show the executor acknowledgment and scope; outcomes beyond what it confirms remain unverified.
Disconnected evidence	Absence of an annotated path means no path was found in this graph version. It does not prove absence of internal use.

Table A.E11: Boundary cases and adjudication rules for the proposed extension.

A reviewer should resolve disagreement by examining the exact source, event identifiers, and context availability, then applying the most specific label whose definition is satisfied. If the evidence remains insufficient, keep an unresolved candidate or no edge. Do not force every node to have a predecessor.

For an interrupted interaction, retain observed content and status. The paper's post-hoc procedure ensures conclusions for its completed dataset traces; this supplement does not impose that requirement on incomplete episodes. A final reconciliation pass may update roles and edges but must not rewrite the source text.

A.E7 Label colors and rendering rules

The tables below preserve the original Appendix A label-fill colors from the supplied PDF. New labels use the operational and provenance palette of the proposed design. The approved legend uses a harmonized UI treatment; it remains the visual reference for boxes and line patterns. Exact fill values below make appendix-style labels reproducible. [1, Appendix A; 2]

Node label	Fill	Node label	Fill
Context	#E0E0E0	Planning	#FFADAD
Fact	#FFD6A5	Reasoning	#FDFFB6
Restatement	#CAFFBF	Assumption	#BDFBDF
Example	#9BF6FF	Reflection	#A0C4FF
Conclusion	#C3B1E1	ToolCall	#CDEFEF
ToolResult	#D9E7FC		

Table A.E12: Original node-label fills and the two proposed operational fills.

Edge label	Fill	Edge label	Fill
infer	#E0E0E0	execute	#D8D8D8
restate	#D0D0D0	elaborate-fact	#C8C8C8
exemplify	#C0C0C0	proceed	#FFCDCD
verify	#FFC4C4	decompose	#FFB9B9
backtrack	#FFADAD	positive	#D0E4FF
uncertain	#C8DBFF	negative	#C0D4FF
support	#FFE5C5	attack	#FFCD8E
invokes	#D8F4F4	returns	#D8F4F4
provides	#D8F4F4	grounded_in	#EEDFF8
cites	#EEDFF8	retry_of	#EEDFF8

Table A.E13: Original edge-label fills and proposed added-edge fills.

Print each label as dark text within its colored box. Use the same rectangular treatment for ToolCall and ToolResult as the original node types. A role name and edge label must remain visible without reliance on hue alone. Put “Provisional,” “Recorded,” or “Inferred” in a separate badge or inspector field; do not change the meaning of a line pattern.

Source references

[1] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-Tür, and Julia Hockenmaier. 2026. ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces. arXiv:2606.05402v1, 3 June 2026. Supplied paper, Appendix A, Tables 3–4; §3; Appendix C.1; Appendix I.

[2] Nathan Conklin and ChatGPT. ReasoningFlow with tool calls and web search: current design discussion and final approved legend, 11 October 2026. Proposed extension. Companion: ReasoningFlowWithToolCall.docx.

A.E8 Annotation record and completion checklist

Store an edge as a versioned annotation with independently auditable evidence. This minimal synthetic record follows the earlier-to-later convention. Its null confidence value avoids implying calibration that has not been established.

```
{
  "source_id": "C2",
  "target_id": "K2",
  "label": "grounded_in",
  "family": "provenance",
  "evidence_method": "annotator_inferred",
  "evidence": {
    "source_id": "manual-S2-text",
    "span_text": "The default timeout is 30 seconds.",
    "context_manifest_ref": "invocation-2"
  },
  "annotation_basis": "The answer uses the cited timeout value.",
  "annotator_confidence": null,
  "state": "committed",
  "version": 1
}
```

A production record should additionally use the immutable source hash and numeric offsets defined in the developer specification. The quoted span above is included for readability; it does not replace source identity.

Check	Completion criterion
Taxonomy	Only the original nine displayed categories plus ToolCall and ToolResult; only the original 14 edges plus the six additions.
Direction	Every committed edge respects earlier-to-later logical order; inverse-reading labels have correctly assigned endpoints.
Identity	Call, attempt, result-item, event, and source identifiers are preserved and do not depend on display text.
Source fidelity	Text spans match the identified immutable representation. Extraction transformations and source availability are recorded.
Evidence origin	Observed associations, explicit references, and inferred semantic judgments remain distinguishable.
Semantic boundaries	No automatic cites→grounded_in, returns→support, or ToolCall→Reasoning conversion.
Graph validity	No cycles, missing endpoints, or self-loops; valid multi-label relations between the same endpoints are retained.
Streaming status	Incomplete units, unresolved references, and interrupted episodes remain visibly provisional or incomplete.
Visual consistency	All nodes are boxes. Label text is boxed and colored. Solid, dashed, and dotted patterns encode family, not confidence.
Reviewability	Each annotation can be checked against exact spans or events; rejected and corrected versions remain auditable.

Table A.E14: Completion checks for manual or automated annotation.

An annotation is complete for a chosen graph version when the observed events are represented, text spans and labels are valid, and unresolved items are explicitly identified. Completion does not mean the graph is a faithful account of hidden model computation or that the underlying answer is correct. The proposed extension requires its own agreement and accuracy evaluation.